

DIP (Dependency Inversion Principle) 1

DIP یکی از اصول پنج‌گانه SOLID است.

♦ تعریف ساده:

ماژول‌های سطح بالا نباید به ماژول‌های سطح پایین وابسته باشند؛ هر دو باید به abstraction مثلاً (interface) وابسته باشند.

♦ تعریف رسمی:

- High-level modules should not depend on low-level modules.
- Both should depend on abstractions.
- Abstractions should not depend on details.
- Details should depend on abstractions.

مثال بد:

```
public class ProductService
{
    private SqlProductRepository _repository = new SqlProductRepository();
}
```

اینجا کلاس سطح بالا مستقیم به کلاس دیتابیس وابسته است

مثال درست با DIP:

```
public interface IProductRepository
{
    List<string> GetProducts();
}

public class ProductService
{
    private readonly IProductRepository _repository;

    public ProductService(IProductRepository repository)
    {
        _repository = repository;
    }
}
```

اینجا وابستگی به interface است ✓

IoC (Inversion of Control) 2

♦ معنی: معکوس شدن کنترل

در حالت عادی، کلاس‌ها خودشان وابستگی‌هایشان را می‌سازند.
در IoC این مسئولیت از کلاس گرفته می‌شود و به بیرون سپرده می‌شود.
به جای اینکه:

```
var service = new ProductService(new SqlProductRepository());
```

سیستم مدیریت وابستگی خودش آن را تزریق می‌کند.

DIP یک اصل طراحی است.

IoC یک مفهوم برای پیاده‌سازی آن اصل است.

IoC Container 3

ابزاری است که:

- وابستگی‌ها را می‌سازد
- آن‌ها را مدیریت می‌کند
- طول عمرشان را کنترل می‌کند

در داتننت معمولاً از کتابخانه Microsoft.Extensions.DependencyInjection استفاده می‌شود.

مثال:

```
var services = new ServiceCollection();  
  
services.AddTransient<IProductRepository, SqlProductRepository>();  
  
var provider = services.BuildServiceProvider();  
var service = provider.GetService<ProductService>();
```

Factory as IoC Container 4

قبل از اینکه IoC Container ها رایج شوند، از الگوی Factory برای ساخت وابستگی استفاده می‌شد. مثال:

```
public class RepositoryFactory
{
    public static IProductRepository Create(string type)
    {
        if (type == "db")
            return new SqlProductRepository();

        if (type == "api")
            return new ApiProductRepository();

        return new FileProductRepository();
    }
}
```

♦ این هم نوعی معکوس کردن کنترل است ولی:

- مدیریت lifecycle ندارد
- پیچیده می‌شود
- مقیاس‌پذیر نیست

Dependency Injection 5

روش عملی پیاده‌سازی DIP است.

وابستگی‌ها از بیرون به کلاس تزریق می‌شوند.

سه نوع اصلی:

1. Constructor Injection (بهترین روش) ✓

2. Property Injection

3. Method Injection

مثال استاندارد:

```
public ProductService(IProductRepository repository)
{
    _repository = repository;
}
```

Service Lifetimes in .NET 6

در ASP.NET Core سه نوع طول عمر داریم:

◆ Transient

هر بار درخواست شود، یک نمونه جدید ساخته می‌شود.

```
services.AddTransient<IMyService, MyService>();
```

◆ Scoped

در هر request یک نمونه ساخته می‌شود.

```
services.AddScoped<IMyService, MyService>();
```

مناسب برای:

- Repository
- DbContext

◆ Singleton

فقط یک بار ساخته می‌شود و در کل برنامه استفاده می‌شود.

```
services.AddSingleton<IMyService, MyService>();
```

جمع‌بندی ارتباط‌ها

1. **IoC** = ایده اصلی وارونگی کنترل.
2. **IoC Container** = ابزار یا چارچوبی که این کنترل را مدیریت می‌کند.
3. **Factory** = نسخه ساده‌ای از IoC Container
4. **Dependency Injection** = روش عملیاتی IoC برای تزریق وابستگی‌ها.
5. **Service Lifetimes** = نحوه مدیریت چرخه عمر اشیاء در Container
6. **DIP** = اصول طراحی که IoC و DI را امکان‌پذیر و مفید می‌کند.